

Wazuh SIEM/XDR Lab

Sfida Extra: deploy del manager e enrollment dell'agent su Kali

Agent: BLACKUTOPIA | Manager: 192.168.56.112 | Epicode

Studente	Giampiero Saponaro
Corso	Cybersecurity & Ethical Hacking, Epicode
Data lab	4 giugno 2026
Manager Wazuh	192.168.56.112 (OVA, rete Host-Only VirtualBox)
Agent	BLACKUTOPIA (Kali Linux 2026.2)
Versione Wazuh	4.14.5

1. Obiettivo della sfida

La traccia chiede di installare Wazuh in versione OVA come piattaforma SIEM/XDR nella rete di laboratorio, collegare il suo agent installato su Kali Linux e interpretare le informazioni raccolte. Cosa: stand-up di un manager Wazuh e di un agent. Perché: simulare uno scenario SOC reale in cui un sensore raccoglie eventi da un endpoint e li centralizza per analisi e compliance. Cosa significa: capire concretamente come un SIEM normalizza log eterogenei (PAM, sudo, dpkg, syscheck) in eventi strutturati con livello di severità, regole, mapping MITRE ATT&CK e correlazione con framework normativi.

2. Architettura del laboratorio

Ho scelto VirtualBox come hypervisor sul mio host Windows 11 Pro. Entrambe le macchine virtuali (la OVA Wazuh e la Kali) sono attaccate alla stessa rete Host-Only vboxnet0 con subnet 192.168.56.0/24, in modo da garantire connettività diretta L3 tra agent e manager senza esporre nulla all'esterno. Il manager espone le porte 1514/TCP (eventi), 1515/TCP (enrollment) e 55000/TCP (API). L'agent sulla Kali utilizza solo 1514 e 1515 in uscita verso 192.168.56.112.

Host	Windows 11 Pro, VirtualBox
Rete VM	Host-Only vboxnet0, 192.168.56.0/24
Wazuh manager	OVA ufficiale wazuh.com, IP 192.168.56.112
Agent endpoint	Kali Linux 2026.2, IP 192.168.56.111, nome BLACKUTOPIA

3. Deploy del manager Wazuh

Ho scaricato l'OVA ufficiale dalla documentazione Wazuh e l'ho importata in VirtualBox. Cosa: import della OVA con 4 vCPU, 8 GB di RAM e 50 GB di disco, adapter di rete su Host-Only. Perché: il manager esegue contemporaneamente OpenSearch (indexer), Wazuh Manager e il dashboard, quindi richiede risorse non banali. Cosa significa: in produzione queste tre componenti vengono spesso separate su nodi diversi; nel mio lab la versione all-in-one OVA è perfetta per studiare il prodotto.

Una volta acceso il manager, mi sono loggato con le credenziali di default wazuh-user e ho recuperato l'IP assegnato dal DHCP della Host-Only. Da Firefox su Kali ho poi raggiunto il dashboard via HTTPS, accettando il certificato self-signed e autenticandomi come admin.

4. Installazione dell'agent su Kali

Ho seguito la modalità APT + Systemd come da documentazione. Cosa: importazione della chiave GPG di Wazuh, aggiunta del repository e installazione del pacchetto wazuh-agent. Perché: il pacchetto Debian gestisce automaticamente la creazione dell'utente di servizio, dei file in /var/ossec e l'unit systemd. Cosa significa: l'agent diventa un servizio gestito che si avvia al boot e si riconnette al manager in caso di disconnessione, comportamento essenziale per un sensore SOC.

```
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | sudo gpg --no-default-keyring \
--keyring gnupg-ring:/usr/share/keyrings/wazuh.gpg --import
sudo chmod 644 /usr/share/keyrings/wazuh.gpg

echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] \
https://packages.wazuh.com/4.x/apt/ stable main" \
| sudo tee /etc/apt/sources.list.d/wazuh.list

sudo apt update
sudo WAZUH_MANAGER="192.168.56.112" WAZUH_AGENT_NAME="BLACKUTOPIA" \
apt install wazuh-agent

sudo systemctl daemon-reload
sudo systemctl enable --now wazuh-agent
```

5. Troubleshooting: ossec.conf rotto

Al primo avvio l'agent falliva immediatamente con il messaggio Error reading XML file 'etc/ossec.conf': (line 0). Cosa: ho ispezionato il file e ho trovato strutture XML non valide (tag <client> orfani fuori dal blocco <ossec_config>, una direttiva <address><192.168.56.112></address> con doppie parentesi angolari, un comando netstat troncato). Perché: tentativi di edit successivi avevano appiccicato configurazioni provenienti da snippet diversi della documentazione, producendo un XML sintatticamente rotto. Cosa significa: questo è un caso scolastico del motivo per cui in produzione si gestiscono i config con tool di Configuration Management (Ansible, Puppet) e si validano con xmllint prima del deploy.

Ho riscritto da zero un ossec.conf minimo e valido:

```
<ossec_config>
<client>
<server>
<address>192.168.56.112</address>
<port>1514</port>
<protocol>tcp</protocol>
</server>
<enrollment>
<enabled>yes</enabled>
<agent_name>BLACKUTOPIA</agent_name>
</enrollment>
</client>
<syscheck>...</syscheck>
<localfile>
<log_format>journald</log_format>
<location>journald</location>
</localfile>
</ossec_config>
```

Una nota importante: Kali moderno non scrive più su /var/log/auth.log e /var/log/syslog ma usa journald. Ho quindi sostituito le due voci syslog standard con un singolo <localfile> di tipo journald, eliminando gli errori Could not open file nei log dell'agent.

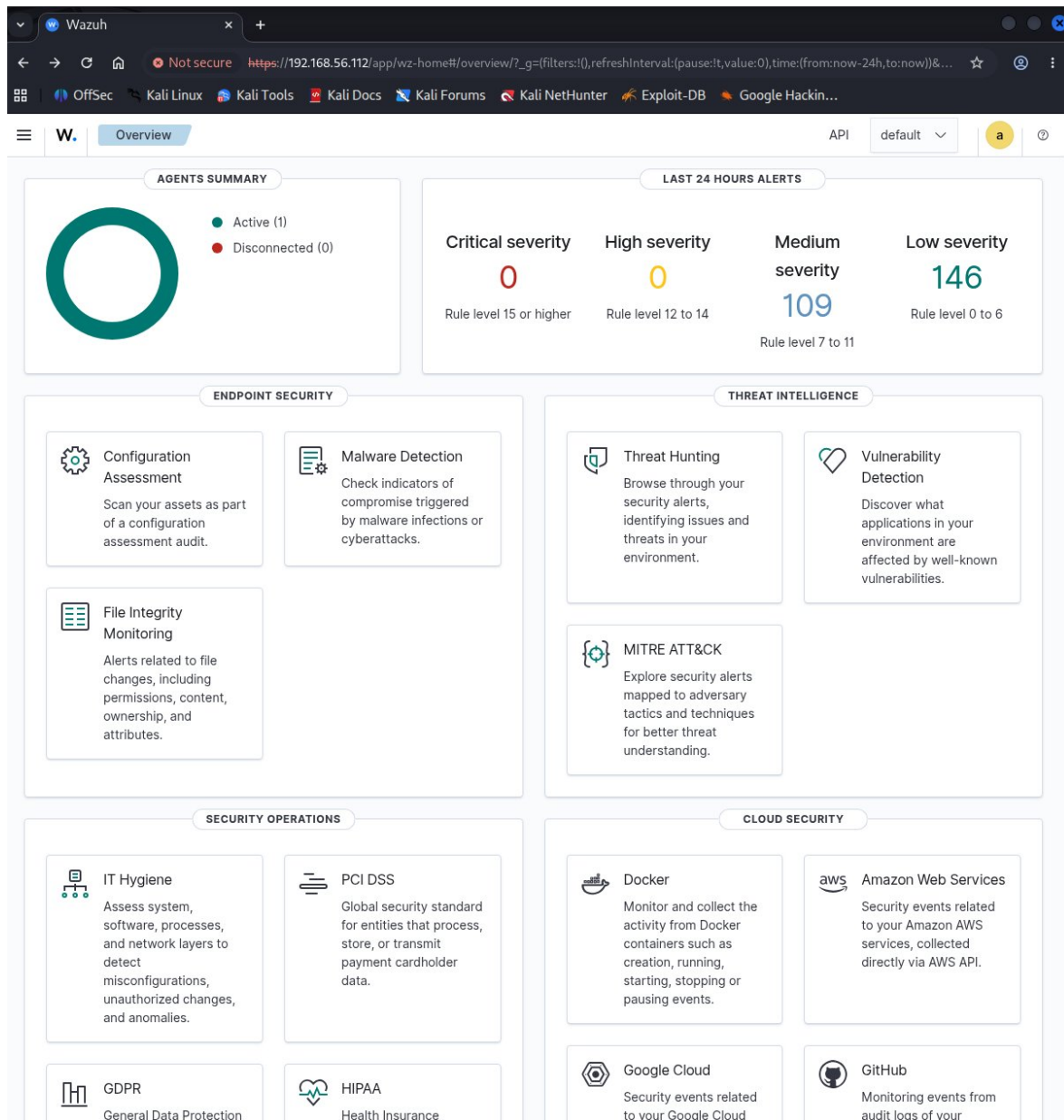
6. Enrollment e prima connessione

Con il config valido e la chiave client azzerata, il primo restart ha innescato l'enrollment automatico. Cosa: l'agent ha contattato il manager sulla porta 1515, ottenuto una chiave AES e si è registrato come BLACKUTOPIA. Perché: senza una chiave condivisa il traffico sulla 1514 verrebbe rifiutato, e questo modello previene la registrazione abusiva di agent. Cosa significa: in un ambiente di produzione si attiva un password per l'enrollment (parametro authd password) per evitare che chiunque sulla rete possa registrare un proprio agent.

```
2026/06/04 12:03:29 wazuh-agentd: INFO: Started (pid: 67945).
2026/06/04 12:03:29 wazuh-agentd: INFO: Requesting a key from server: 192.168.56.112
2026/06/04 12:03:29 wazuh-agentd: INFO: Using agent name as: BLACKUTOPIA
2026/06/04 12:03:29 wazuh-agentd: INFO: Valid key received
2026/06/04 12:03:30 wazuh-syscheckd: INFO: (6000): Starting daemon...
2026/06/04 12:06:00 wazuh-logcollector: INFO: (9203): Monitoring journal entries.
```

7. Overview del dashboard

Dopo qualche minuto di esercizio il dashboard ha popolato la sezione Overview con i numeri della prima sessione di telemetria.



Wazuh Overview: 1 agent attivo (BLACKUTOPIA), 0 critical, 0 high, 109 medium severity, 146 low severity nelle prime ore.

Cosa: 255 alert totali in poche ore di vita dell'agent. Perché: Kali è una distribuzione offensiva con decine di tool non hardened; il primo scan SCA fa scattare molte regole di compliance, e i comandi che ho lanciato volontariamente per stressare il sensore hanno aggiunto eventi di authentication failure, sudo, useradd e dpkg. Cosa significa: un agent appena collegato a un endpoint Linux non-hardened produce sempre rumore iniziale. In un SOC reale questo è il momento del tuning, in cui si creano white-list e si calibra il livello minimo degli alert per ridurre i falsi positivi.

8. Generazione e analisi degli eventi

Per popolare il sensore con eventi controllati ho eseguito sulla Kali una sequenza di comandi che coprono diverse categorie di rilevamento.

```
# 1. Authentication failed via SSH
ssh nonesiste@localhost

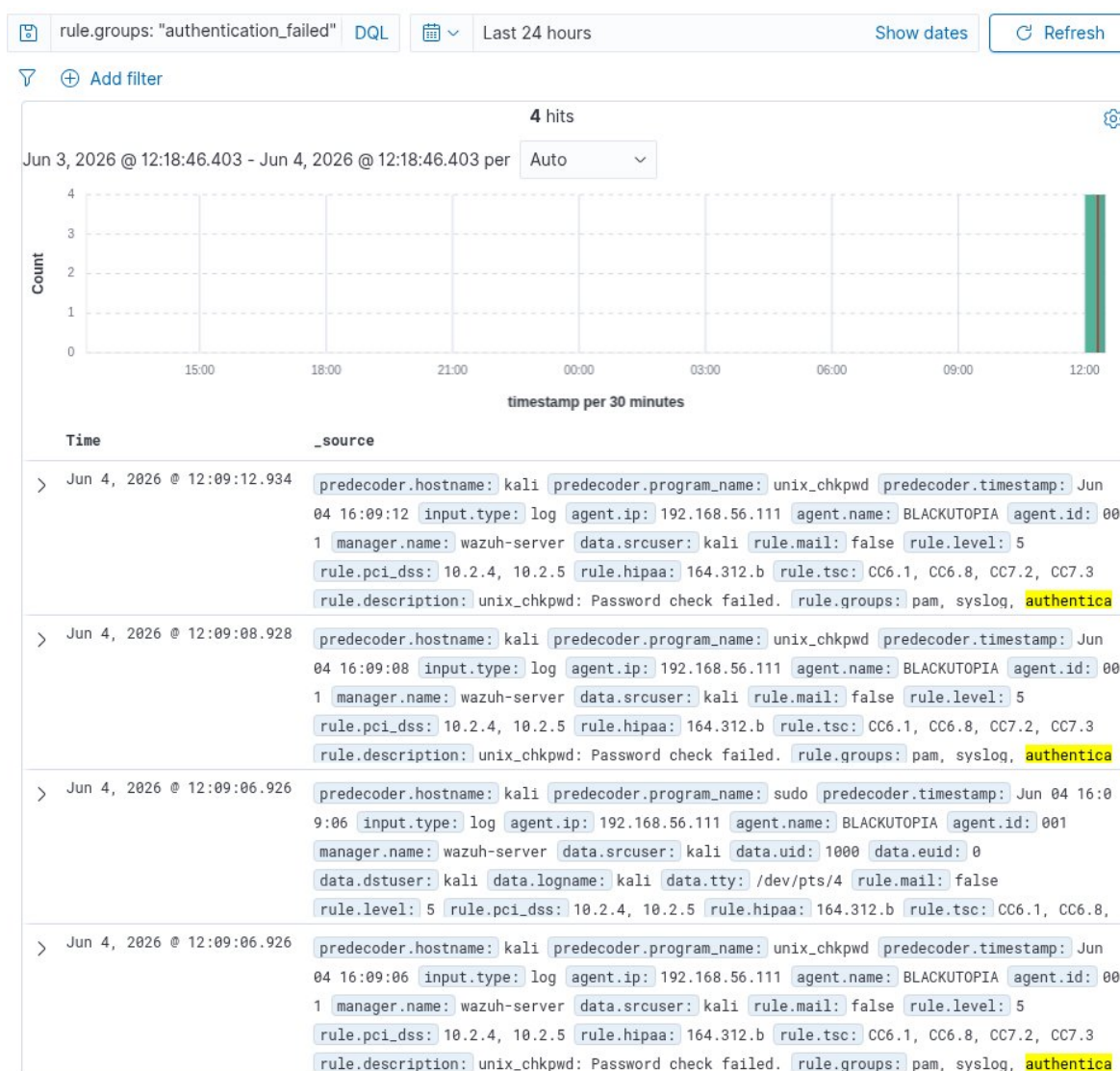
# 2. Sudo con password errata, poi corretta
sudo -k
sudo whoami

# 3. File Integrity Monitoring
sudo bash -c 'echo "# test wazuh" && /etc/motd'

# 4. User lifecycle
sudo useradd test_intruder
sudo userdel test_intruder

# 5. Package installation
sudo apt install -y figlet
```

8.1 Authentication failed



Filtro rule.groups: "authentication_failed" - 4 hit con unix_chkpwd Password check failed, level 5.

Cosa: il decoder PAM ha intercettato il modulo `unix_chkpwd` via `journald` e ha matchato la rule del gruppo `pam`, `syslog`, `authentication_failed`. Perché: ogni tentativo fallito su `sudo` passa attraverso `unix_chkpwd`, che produce un evento `Password check failed`; il decoder estrae il campo `data.srcuser` con il nome dell'utente che ha provato. Cosa significa: questo è il pattern che in un SOC permette di rilevare un brute-force locale o un'escalation tentata da un utente compromesso. Le pillole `rule.pci_dss: 10.2.4/10.2.5` e `rule.hipaa: 164.312.b` mostrano che ogni evento è già mappato sui framework di compliance, materiale prezioso in fase di audit.

8.2 Eventi a media e alta severità



Filtro `rule.level >= 7` - 109 hit. In evidenza `useradd test_intruder` (level 8), `sudo whoami` a root (level 10) e i risultati SCA con score 45/190.

Cosa: tre famiglie di eventi distinte. Il primo gruppo è la coppia `useradd/userdel` con livello 8 e i metadati completi (`data.uid: 1001`, `data.home: /home/test_intruder`, `data.shell: /bin/sh`). Il secondo è il `Successful sudo to ROOT executed` con livello 10, che riporta `data.srcuser: kali`, `data.dstuser: root`, `data.tty: pts/4`, `data.pwd: /home/kali` e `data.command: /usr/bin/whoami`. Il terzo gruppo sono i risultati del modulo SCA

con `data.sca.score`: 45 su 190 check totali e 6 invalid, basati sulla policy `sca_distro_independent_linux.yml`. Perché: il livello 10 è la soglia tipica usata dai SOC per generare un ticket; un sudo a root da un utente standard è sempre un evento da tracciare anche quando legittimo. Cosa significa: il SIEM non racconta solo che è successo qualcosa, ma fornisce chi, cosa, dove e quando con campi normalizzati, pronti per la correlazione e per il triage da parte di un analista.

8.3 Eventi mappati su MITRE ATT&CK



Filtro `rule.mitre.id: *` - 32 hit. PAM Login session opened, apt install figlet, userdel test_intruder, sudo /usr/sbin/userdel.

Cosa: 32 eventi sono già annotati con tecniche MITRE ATT&CK. Vedo i comandi `/usr/bin/apt install -y figlet` e `/usr/sbin/userdel test_intruder` con il loro mapping. Perché: Wazuh include un dizionario di rules che associa decoder e tecniche ATT&CK; questo permette di costruire dashboard di copertura difensiva per tattica (Initial Access, Execution, Persistence, ecc). Cosa significa: in un ambiente di threat hunting reale, partire dalla matrice MITRE permette di rispondere alla domanda quali tecniche d'attacco sono visibili sui miei sensori e quali invece sono cieche. È il primo passo per costruire un programma di detection engineering.

9. Riepilogo regole osservate

Mappa di corrispondenza fra le azioni che ho eseguito e le regole Wazuh che ne hanno generato gli alert.

Comando lanciato	Rule ID	Level	Descrizione regola	Gruppo
sudo whoami (password errata)	5503	5	unix_chkpwd: Password check failed	pam, syslog, authentication_failed
sudo whoami (password corretta)	5402	10	Successful sudo to ROOT executed	pam, syslog, sudo
useradd test_intruder	5902	8	New user added to the system	adduser, syslog
userdel test_intruder	5904	8	Group or user deleted from the system	syslog, useradd
apt install figlet	2902	3	New dpkg (Debian Package) installed	syslog, dpkg
Sessione SSH/sudo aperta	5501	3	PAM: Login session opened	pam, syslog

10. Considerazioni e prossimi passi

La sfida mi ha fatto toccare con mano la differenza fra un raccoglitore di log e un SIEM. Un raccoglitore mi avrebbe consegnato righe di `/var/log/auth.log`; Wazuh mi consegna eventi con livello, gruppo, tecnica MITRE, mapping di compliance e arricchimento dei campi. Quello che configurerei in più, avendo più tempo:

Vulnerability Detector: per ora la sezione Vulnerabilities è poco popolata; abilitando i feed CVE nel manager Wazuh metterebbe in croce CVE-by-CVE i pacchetti di Kali, e ne troverebbe parecchi viste le librerie offensive incluse. Integrazione VirusTotal: agganciando i hash dei file modificati al lookup VirusTotal otterrei alert immediati su binari noti come malevoli. Custom rules: scrivere regole locali per matchare pattern specifici dei miei lab CTF (per esempio shell inverse note, comandi tipici di privilege escalation). Active Response: il modulo è abilitato lato config ma non l'ho ancora collegato a script di mitigazione; potrei usarlo per bloccare automaticamente un IP sorgente dopo N authentication failed.

L'esperienza del troubleshooting iniziale sull'`ossec.conf` rotto, fastidiosa sul momento, è stata didatticamente la parte più utile della sfida: ho capito da dove parte la pipeline (un singolo file di configurazione XML), come la rompi velocemente con un copia-incolla incompleto, e come la valida prima di metterla in produzione (`xmllint`, audit del file).